

● 課程專題(2)：FPGA Lab

➤ ALTERA / Quartus 燒錄：

(詳細模擬與單次執行步驟可參考總資料夾)

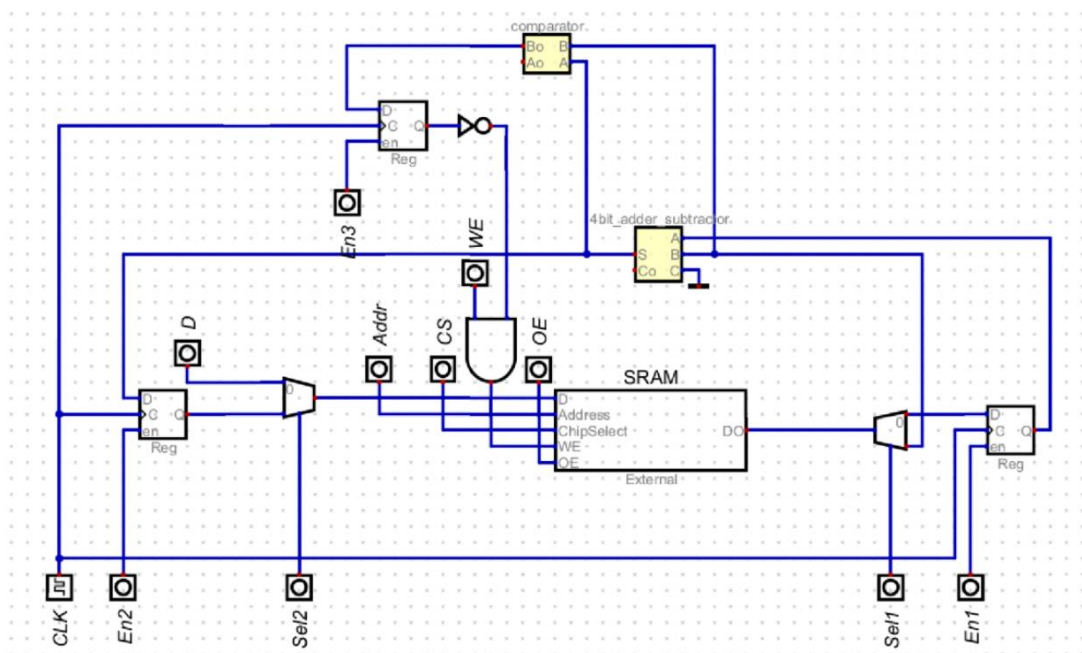
一、目的：

取自數位邏輯實習的其中一個 Lab，本課程為兩人一組分題進行；SRAM 累加器為我負責項目（電路設計、VHDL、Digital/Quartus 模擬、Altera 開發板燒錄與實機驗證）。這個題目橫跨兩次實作，它記錄了我把手動操作的資料路徑轉換為狀態機自動控制，最後燒錄進 FPGA 的完整過程。

二、系統架構：

手動階段：

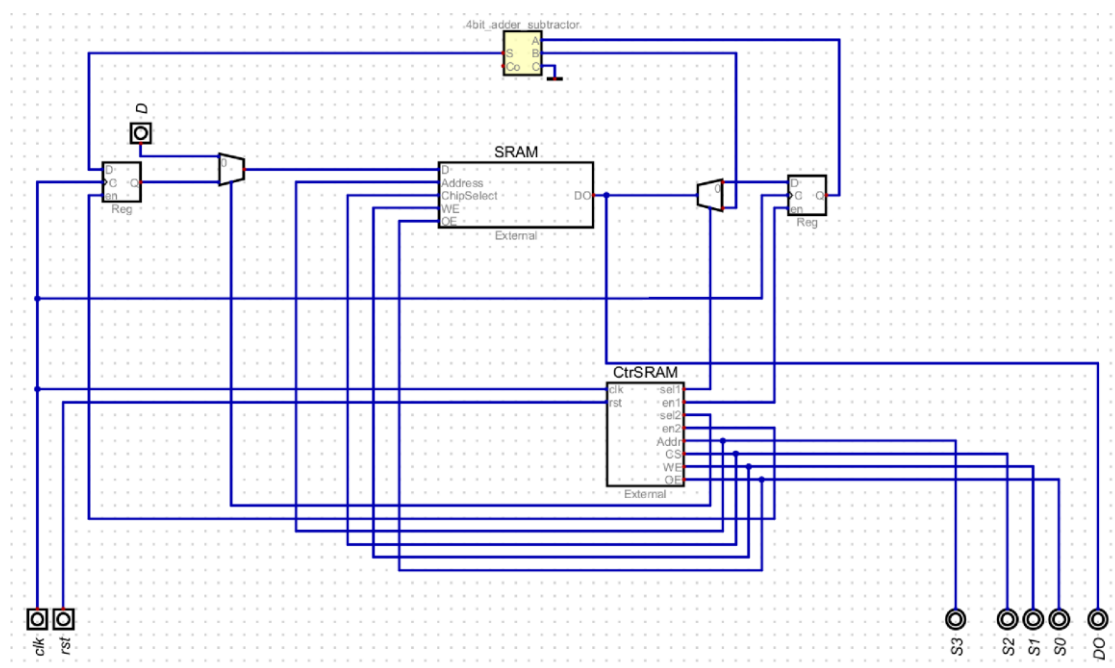
我先以 VHDL 實作一顆 4×4 SRAM（以二維陣列描述記憶體，CS/WE/OE 分別控制晶片致能、寫入與讀出，未致能時輸出高阻抗），再把它與我先前實作的四位元加減法器、比較器整合成一個可偵測溢位的累加器（圖二十二）。此版本用多組 enable/select 線，逐步操作「讀出 SRAM → 經加法器相加 → 存回暫存器 → 寫回 SRAM」，並用比較器判斷溢位（累加後變小即代表溢位），溢位時鎖住 SRAM。



圖(二十二) SRAM Accumulator VHDL 模擬電路 (手動控制)

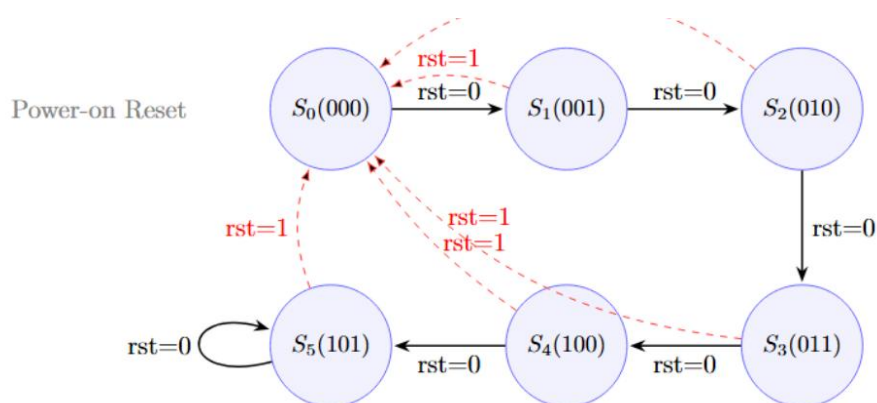
自動階段：

我沿用同一顆 SRAM 與加減法器，把上一版整排手動控制線整合成一個 Moore 有限狀態機，從 S_0 (reset 初始) 依序走 $S_1 \sim S_5$ ，最後停在 S_5 反覆累加，並加入同步 reset 一鍵清零 (圖二十三)。



圖(二十三) SRAM Accumulator VHDL 模擬電路 (狀態機控制)

隨著 Clock 進行動作，並且加入 reset 訊號，當 $\text{reset} = 1$ 時會將 SRAM 與當前控制狀態清 0 為初始(圖二十四)。



圖(二十四) SRAM Accumulator state graph

下表列出六個狀態對應的完整控制輸出(表五)，由於採 Moore 設計每個狀態的輸出固定，控制線又分成 Datapath Control 與 SRAM Bus 兩群，SRAM Bus 在六個狀態下的組合彼此互異，沒有重複，因此用於對應狀態的可觀測指示（Address 雖為 2 bits 但只用到 0/1，最高位恆為 0，判讀時從次一位看起）。

表(五) SRAM Accumulator state table

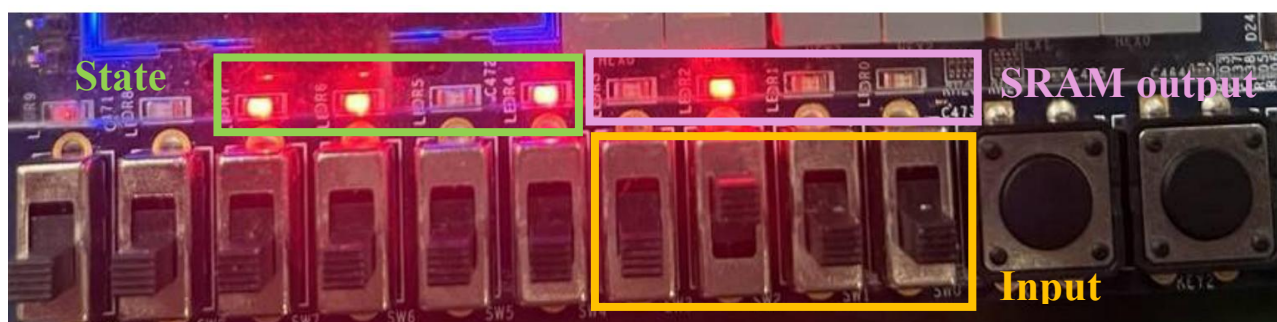
State	Encoding ($Q_2Q_1Q_0$)	Datapath Control (A, B, C, D)				SRAM Bus (E)			
		sel1	en1	sel2	en2	[Addr	CS	WE	OE]
S_0	000	0	0	0	0	[00	0	0	0]
S_1	001	0	0	0	0	[00	1	1	0]
S_2	010	0	0	0	0	[01	1	1	0]
S_3	011	0	1	0	0	[00	1	0	1]
S_4	100	1	0	1	1	[01	1	0	1]
S_5	101	1	0	1	1	[01	1	1	1]

三、 驗證與結果：

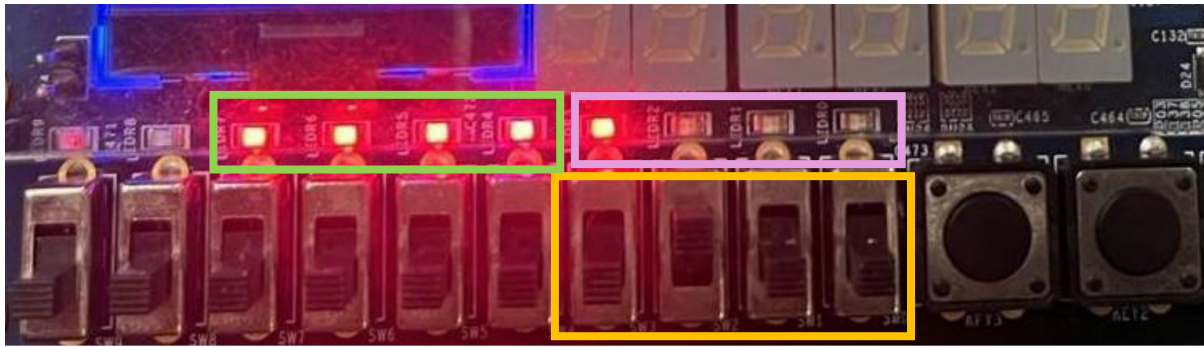
用 Quartus 把設計燒錄到 Altera 開發板，以指撥開關輸入，LED 顯示狀態與累加結果，實機跑完一輪完整的累加序列（每輪 +4）並驗證溢位保護。累加步驟與結果如下表(表六)。

表(六) 累加序列與結果

累加輪次	SRAM 輸出	累加結果	狀態	實機結果
初始 (reset)	Z	0	S_0	SRAM 清零、LED 全滅
第 1 輪 (+4)	4	0(下個 CLK 輸入)	S_1-S_3	初始存入 SRAM
第 2 輪 (+4)	4	8 (4+4)	S_4	累加正確 (圖)
第 3 輪 (+4)	8	C (12)	S_5	累加正確 (圖)
第 4 輪 (+4)	C (12)	溢位	S_5	偵測溢位、SRAM 鎖住 (圖)



圖(二十五) SRAM 輸出 4 (狀態 S_4 ，累加器為 8)



圖(二十六) SRAM 輸出 8(狀態 S5，累加器為 12)

當累加超過 4-bit 上限，比較器判定溢位、控制邏輯立即鎖住 SRAM 寫入，避免錯誤資料寫入，機制如預期運作，reset 後即可重新開始累加。



圖(二十七) SRAM 輸出 12 (狀態 S5，累加器清空)

四、 技能與其他作品關聯

也因此我才會認為雙核心 CPU 專題中設計已具備上板條件，我不只會在模擬器裡驗證，也真的用過 Quartus 把 VHDL 燒進 Altera FPGA。

後續也有使用過 Xilinx 與計算機組織的課程模擬等，也證明了我具備 VHDL/Verilog 兩種語言、並實際操作過 Altera 與 Xilinx 兩大 FPGA 工具鏈的數位設計經驗。

➤ 其他環境 (Xilinx ISE)：

(模擬檔案可參考總資料夾)

本節資料取自「FPGA 系統設計實務」與「計算機組織」兩門課，開發板為 Xilinx Spartan-6 FPGA SP601，由於未錄下課程實機執行畫面，以下以只需模擬的題目進行呈現。

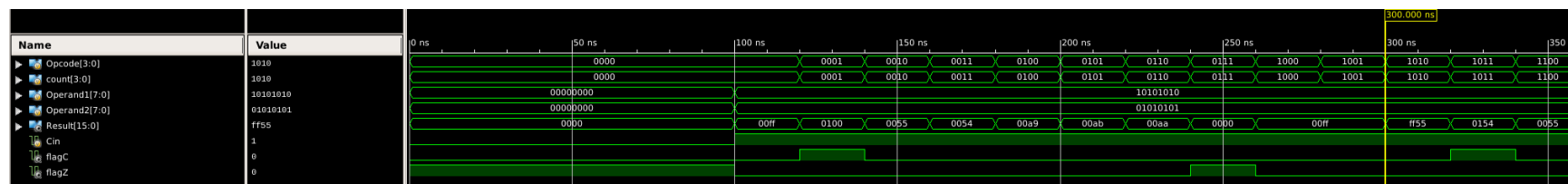
一、 目的：

我在雙核心 CPU 專題中雖已實作過一顆最小的 8-bit ALU，搭配計算機組織課程所學，以及 FPGA 系統設計實務所架設之環境，更完整地理解運算單元的設計以及工具使用，內容取自計算機組織作業，打造一顆支援 13 種運算的 8-bit ALU。

二、 設計概述與結果驗證：

於 Xilinx ISE 以 Behavioral Description 模擬 13 功能 8-bit ALU：輸入為 4-bit Opcode、兩組 8-bit 運算元與 Cin，輸出 16-bit Result 與 flagC／flagZ，共支援 13 種運算。

此 ALU 為純組合邏輯，Opcode 或輸入一變，Result 與旗標即在延遲後更新。測試以迴圈掃過全部 13 種運算（固定 Operand1 = AA、Operand2 = 55、Cin = 1），並以模擬驗證（圖二十八）。



圖(二十八) 8-bit ALU 於 Xilinx ISim 的功能模擬

Opcode 掃過 13 種運算，Result 與 flagC／flagZ 皆符合預期 (表七)。

表(七) 全部 13 種運算模擬驗證 (Operand1 = AA、Operand2 = 55、Cin = 1)

Opcode	運算	運算式	Result (hex)	flagC / flagZ
0000	ADD	AA + 55	00FF	0 / 0
0001	ADDC	AA + 55 + Cin	0100	1 / 0
0010	SUB	AA - 55	0055	0 / 0
0011	SUBB	AA - 55 - Cin	0054	0 / 0
0100	DEC	AA - 1	00A9	0 / 0
0101	INC	AA + 1	00AB	0 / 0
0110	TRA	Operand1 (傳遞)	00AA	0 / 0
0111	AND	AA & 55	0000	0 / 1
1000	OR	AA 55	00FF	0 / 0
1001	XOR	AA ^ 55	00FF	0 / 0
1010	NOT	~AA	FF55	0 / 0
1011	SHL	AA << 1	0154	1 / 0
1100	SHR	AA >> 1	0055	0 / 0

三、 總結：

綜合以上子專案，我具備使用 VHDL 與 Verilog 兩種硬體描述語言、Altera Quartus 與 Xilinx ISE 兩大 FPGA 工具鏈經驗，而因為 SP601 屬 Spartan-6 舊世代，只能以已停止維護的 ISE 工具鏈開發，而 ISE 在新版作業系統上常安裝困難。我因此以 Oracle VirtualBox 架設相容環境來執行 ISE，也讓我對於使用虛擬機等環境有更多了解。